编程实验 4

题目：

设从键盘输入一序列的字符 a1,a2,a3…an。设计一个算法实现这样的功能：若 ai 为数字字符， ai 进队；若 ai 为小写字母，将队首元素出队；若 ai 为其他字符，表示输入结束。要求：使用循环队列和链队列两种方式实现。

源程序：

循环队列实现：



```c
/*刘超然-2023060904025-循环队列*/
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX_SIZE 100  // 队列的最大容量

// 定义循环队列
typedef struct {
    char data[MAX_SIZE];  // 队列数组
    int front;            // 队头指针
    int rear;             // 队尾指针
} CircularQueue;

// 初始化队列
void initQueue(CircularQueue* q) {
    q->front = 0;
    q->rear = 0;
}

// 判断队列是否为空
int isEmpty(CircularQueue* q) {
    return q->front == q->rear;
}

// 判断队列是否已满
int isFull(CircularQueue* q) {
    return (q->rear + 1) % MAX_SIZE == q->front;
}

// 入队操作
void enqueue(CircularQueue* q, char c) {
    if (isFull(q)) {
        printf("队列已满，无法进队\n");
    }
    else {
        q->data[q->rear] = c;
        q->rear = (q->rear + 1) % MAX_SIZE;
    }
}

// 出队操作
char dequeue(CircularQueue* q) {
    if (isEmpty(q)) {
        printf("队列为空，无法出队\n");
        return '\0';
    }
    else {
        char c = q->data[q->front];
        q->front = (q->front + 1) % MAX_SIZE;
        return c;
    }
}

// 打印队列中的元素
void displayQueue(CircularQueue* q) {
    if (isEmpty(q)) {
        printf("队列为空\n");
        return;
    }

    int i = q->front;
    printf("队列中的元素：");
    while (i != q->rear) {
        printf("%c ", q->data[i]);
        i = (i + 1) % MAX_SIZE;
    }
    printf("\n");
}

// 处理输入字符
void processInput(char sequence[]) {
    CircularQueue q;
    initQueue(&q);
    char dequeuedElements[MAX_SIZE];
    int dequeueCount = 0;

    for (int i = 0; sequence[i] != '\0'; i++) {
        char ch = sequence[i];
        if (isdigit(ch)) {  // 如果是数字字符，进队
            enqueue(&q, ch);
        }
        else if (islower(ch)) {  // 如果是小写字母，出队
            char dequeued = dequeue(&q);
            if (dequeued != '\0') {
                dequeuedElements[dequeueCount++] = dequeued;
            }
        }
        else {  // 其他字符，结束输入
            break;
        }
    }

    // 输出队列中的元素
    displayQueue(&q);

    // 输出出队的元素
    printf("出队的元素：");
    for (int i = 0; i < dequeueCount; i++) {
        printf("%c ", dequeuedElements[i]);
    }
    printf("\n");
}

int main() {
    char sequence[100];
    printf("请输入字符序列(以非数字和小写字母作为结束)：");
    fgets(sequence, 100, stdin);

    processInput(sequence);

    return 0;
}
```

链队列实现：

循环队列.c　链队列.c

计算机软件基础-实验-4　(全局范围)

```c
/*刘超然-2023060904025-链队列*/
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

typedef struct Node {
    char data;
    struct Node* next;
} Node;

// 定义链队列
typedef struct {
    Node* front;
    Node* rear;
} LinkedQueue;

// 初始化队列
void initQueue(LinkedQueue* q) {
    q->front = q->rear = NULL;
}

// 判断队列是否为空
int isEmpty(LinkedQueue* q) {
    return q->front == NULL;
}

// 入队操作
void enqueue(LinkedQueue* q, char c) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (!newNode) {
        printf("内存分配失败\n");
        return;
    }
    newNode->data = c;
    newNode->next = NULL;

    if (isEmpty(q)) {
        q->front = q->rear = newNode;
    }
    else {
        q->rear->next = newNode;
        q->rear = newNode;
    }
}

// 出队操作
char dequeue(LinkedQueue* q) {
    if (isEmpty(q)) {
        printf("队列为空，无法出队\n");
        return '\0';
    }
    else {
        Node* temp = q->front;
        char c = temp->data;
        q->front = q->front->next;
        free(temp);
        return c;
    }
}

// 打印队列中的元素
void displayQueue(LinkedQueue* q) {
    if (isEmpty(q)) {
        printf("队列为空\n");
        return;
    }

    Node* temp = q->front;
    printf("队列中的元素: ");
    while (temp != NULL) {
        printf("%c ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// 处理输入字符
void processInput(char sequence[]) {
    LinkedQueue q;
    initQueue(&q);
    char dequeuedElements[100];
    int dequeueCount = 0;

    for (int i = 0; sequence[i] != '\0'; i++) {
        char ch = sequence[i];
        if (isdigit(ch)) {  // 如果是数字字符，进队
            enqueue(&q, ch);
        }
        else if (islower(ch)) {  // 如果是小写字母，出队
            char dequeued = dequeue(&q);
            if (dequeued != '\0') {
                dequeuedElements[dequeueCount++] = dequeued;
            }
        }
        else {  // 其他字符，结束输入
            break;
        }
    }

    // 输出队列中的元素
    displayQueue(&q);

    // 输出出队的元素
    printf("出队的元素: ");
    for (int i = 0; i < dequeueCount; i++) {
        printf("%c ", dequeuedElements[i]);
    }
    printf("\n");
}

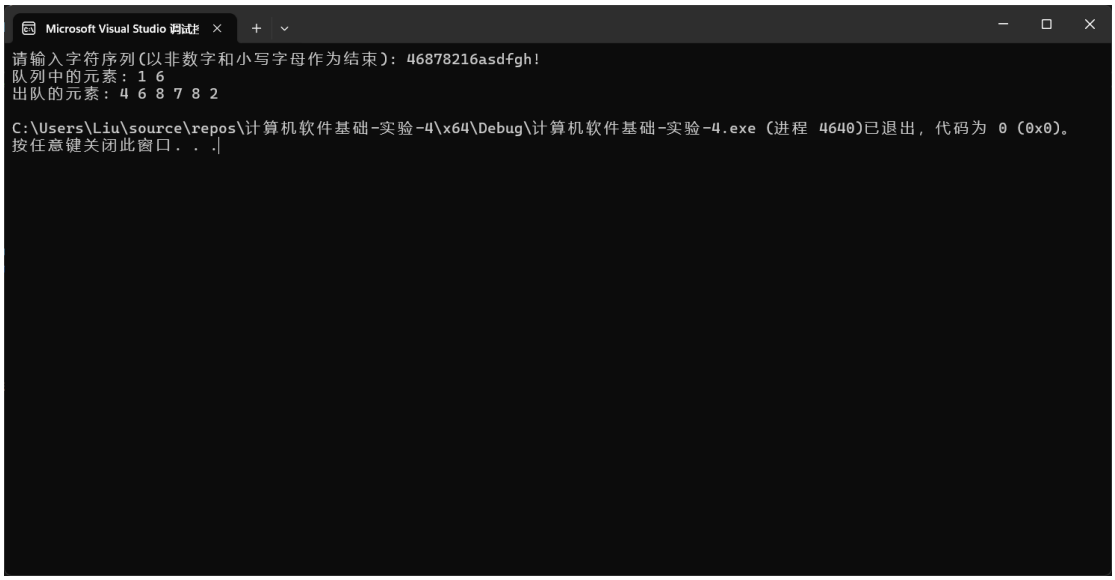
int main() {
    char sequence[100];
    printf("请输入字符序列(以非数字和小写字母作为结束): ");
    fgets(sequence, 100, stdin);

    processInput(sequence);

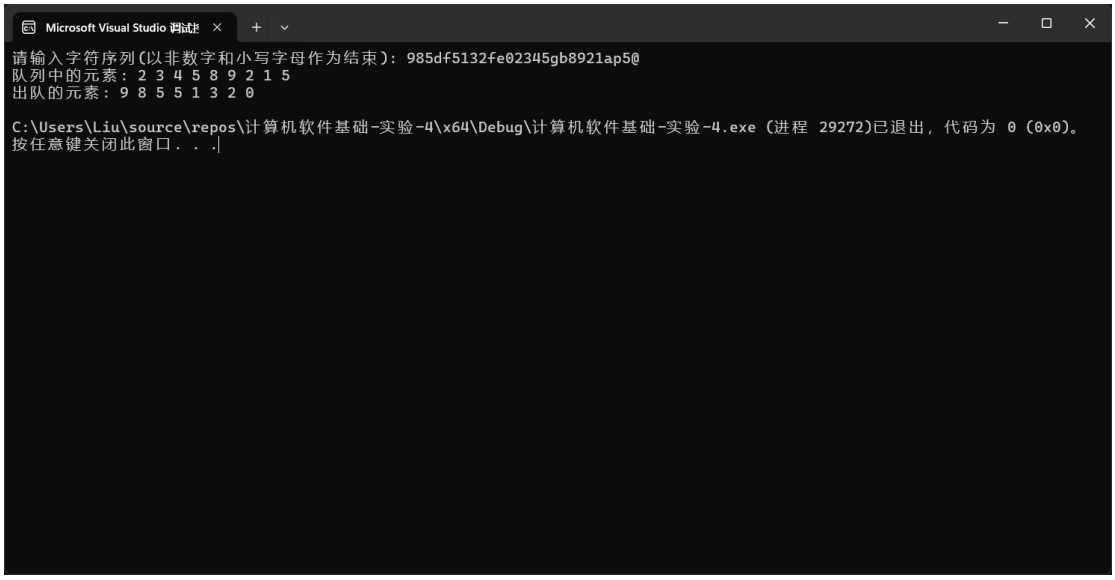
    return 0;
}
```

运行结果：

循环队列实现：



链队列实现：



心得体会：

在实现循环队列时，我学会了如何使用数组和指针来管理队列的进出队操作，解决了传统队列因队头出队导致的空间浪费问题。循环队列的限制在于需要预先设定队列的大小，适用于队列容量已知的场景。

链队列的实现则让我进一步理解了链表的动态内存分配。通过链表结构，我能够在不预设大小的情况下灵活地管理队列元素，适用于元素数量不固定的情况。链队列的实现也让我加深了对指针操作和内存管理的理解。

这次实验帮助我掌握了队列的基本操作，我对队列数据结构有了更深入的理解，特别是循环队列和链队列的区别与应用，并通过实际编程提高了解决问题的能力。不同队列结构的对比让我意识到，在实际开发中需要根据具体需求选择合适的数据结构。